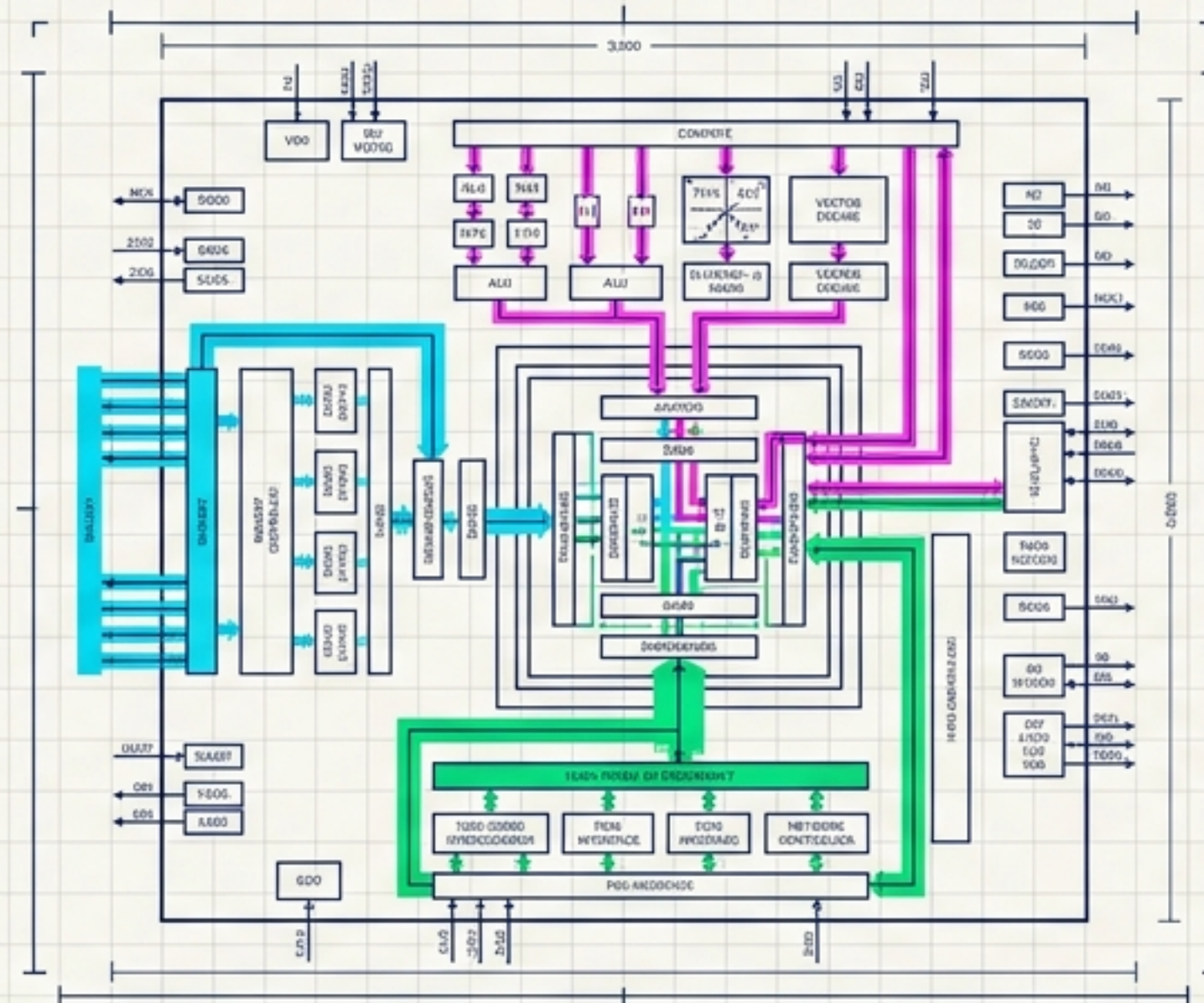


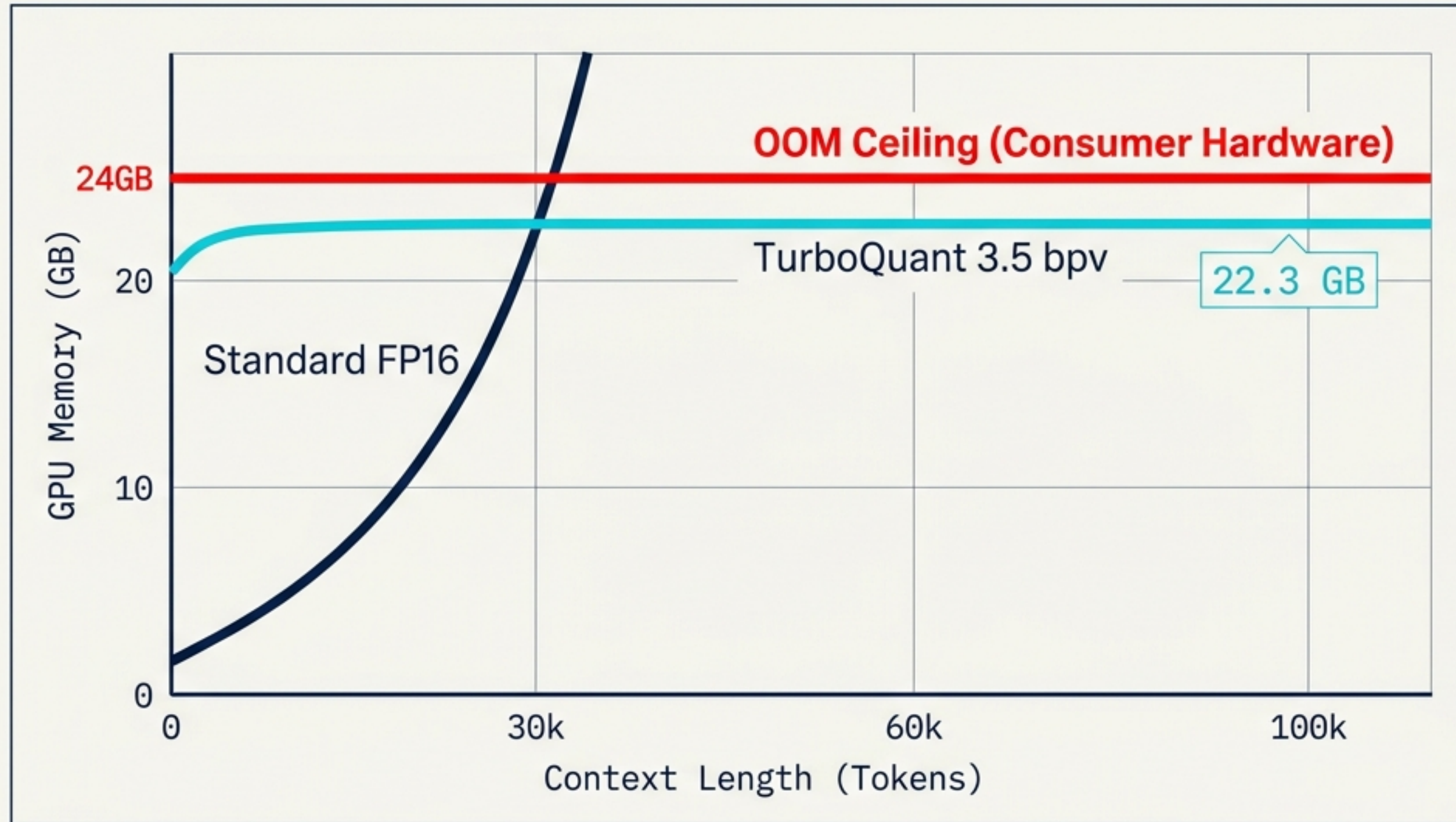
The Local Frontier

Engineering the 100B+ Parameter Local AI Stack



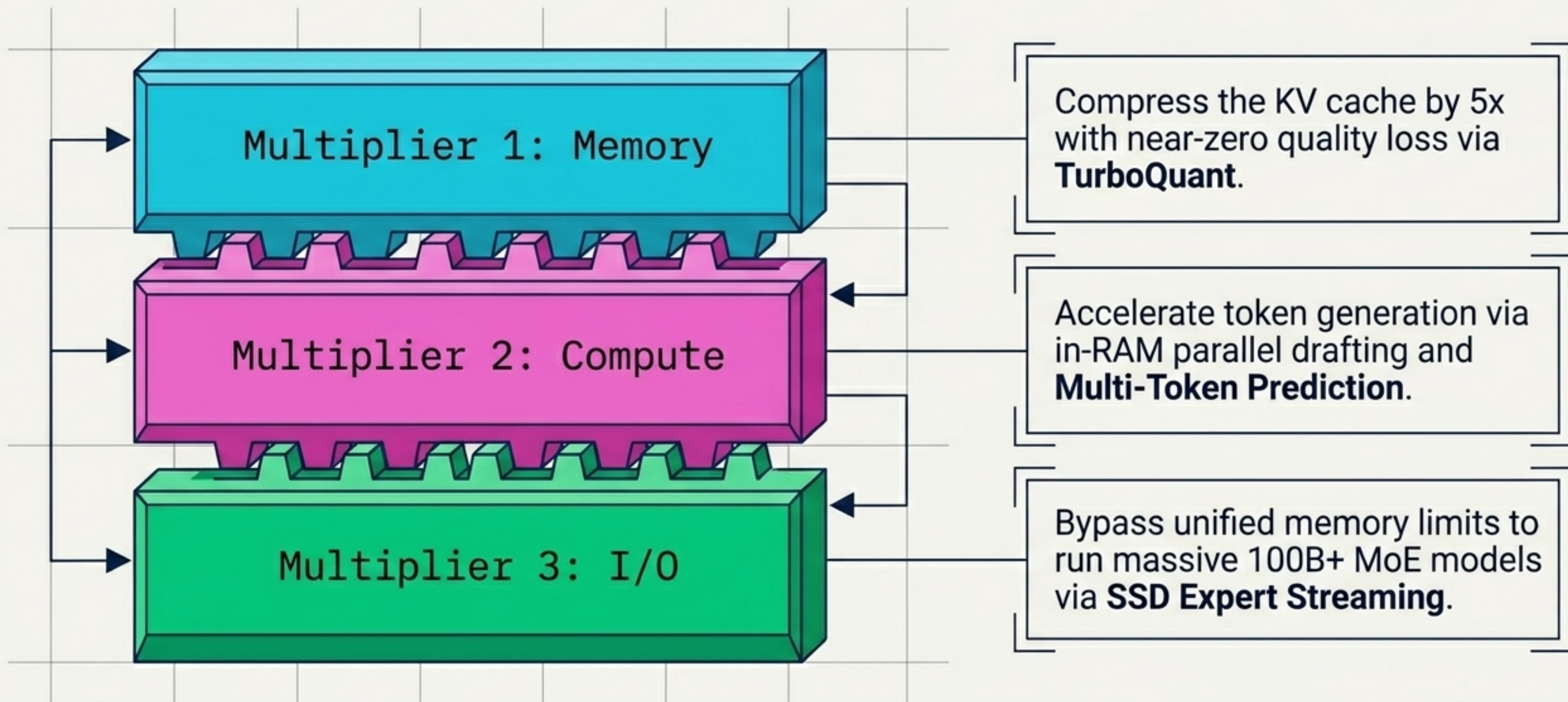
VERSION_2026.4 // KV_COMPRESSION // SPEC_DECODING // SSD_STREAMING

Linear context scaling destroys local VRAM



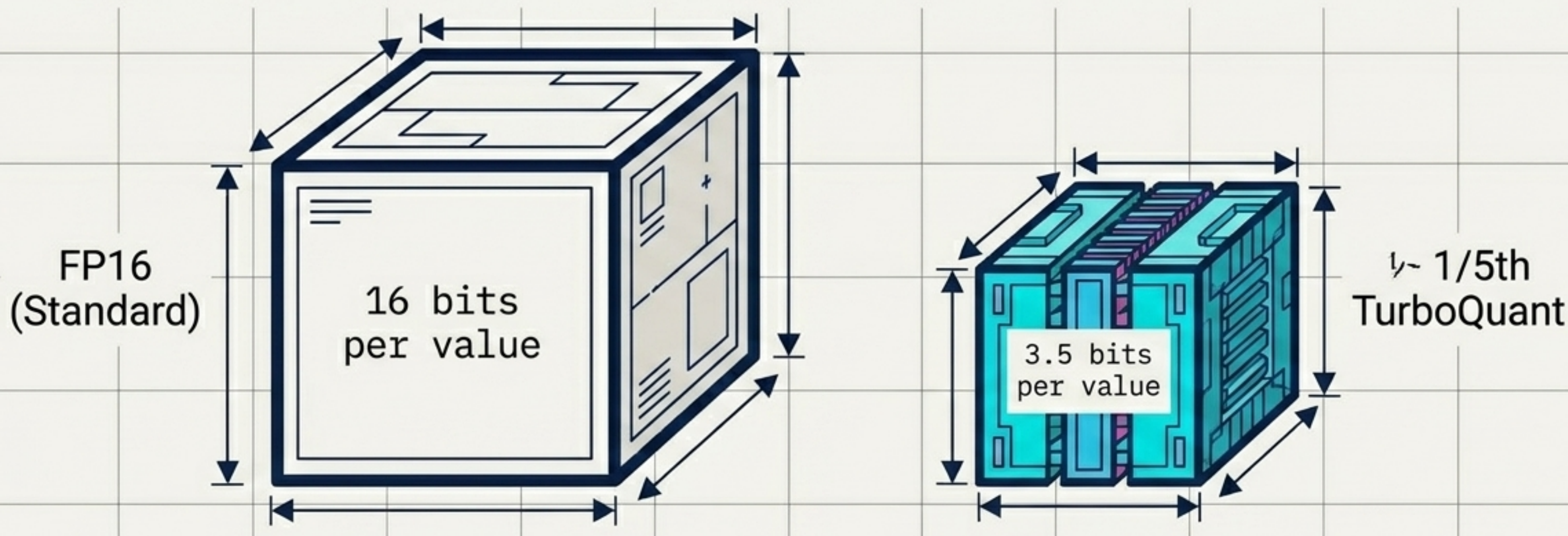
The shift from cloud dependency to local frontier models requires fundamentally severing the linear relationship between context length and hardware memory.

Cracking the VRAM wall requires stackable multipliers



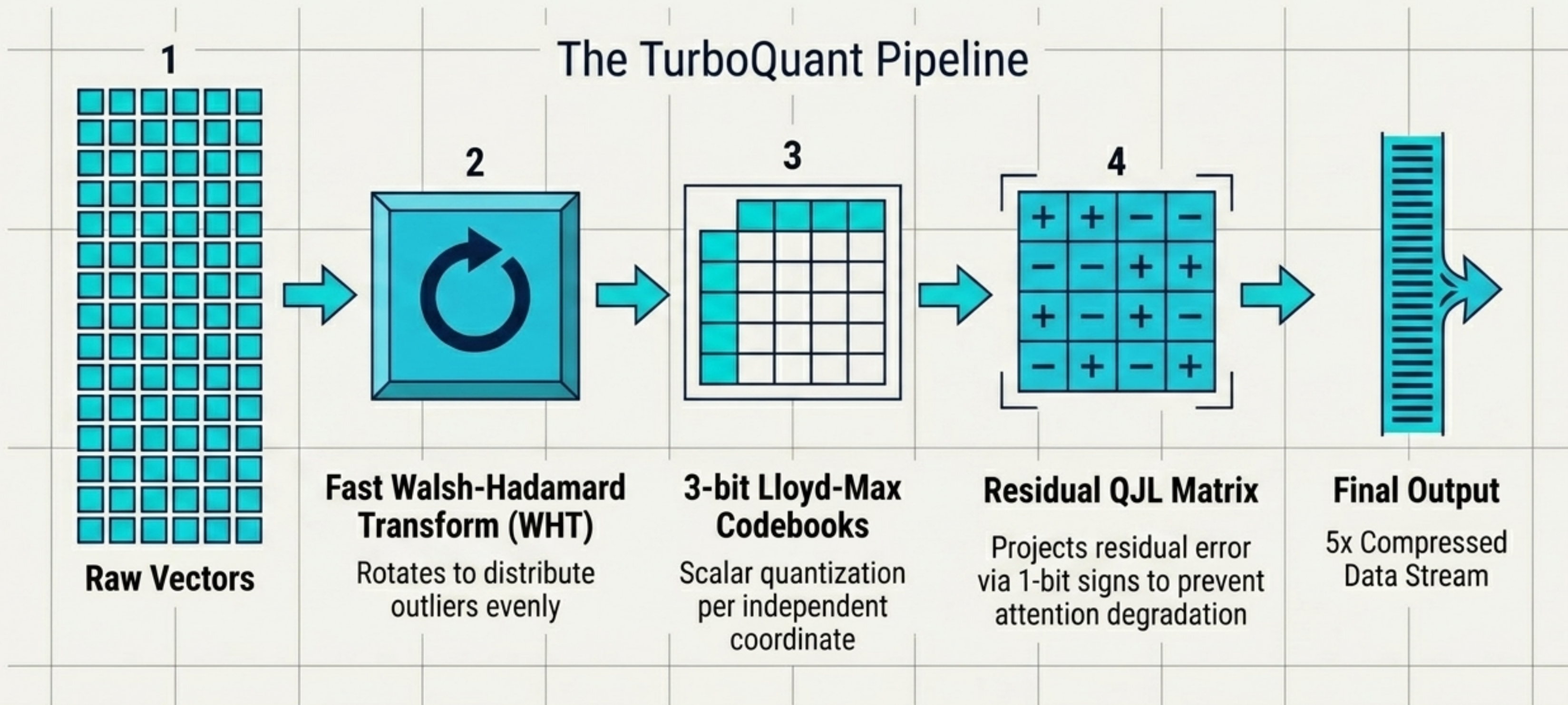
TurboQuant achieves V3 quality at V2 speeds

By compressing the KV cache to ~3.5 bits per coordinate overall, TurboQuant delivers 5x compression with near-identical quality to FP16. It avoids the deep model error compounding found in recursive polar transforms.



Mistral-7B-Instruct-v0.3 || Top-1 Match: 80% || Compression: 4.9x

Independent coordinate quantization eliminates memory overhead



Precision compression stacks cleanly with token selection

The 2026 KV Compression Landscape Matrix

Method	Axis	Impact	Rule
TurboQuant	Precision (3.5 bpv)	4.9x reduction	The Baseline Container
TriAttention	Token Selection	10.7x reduction on 32K CoT	Orthogonal – Stack it
SnapKV / H20	Token Eviction	4–20x reduction	Orthogonal – Stack it
NVFP4 KV	Hardware Container	Native Blackwell format	TQ lives inside NVFP4 blocks

Multi-Token Prediction converts sequential bottlenecks into bulk verification

Yields **1.81x** average throughput and nearly **2x** faster Time-To-First-Token (TTFT) at 100K context with **zero** additional memory overhead.

Standard Autoregressive



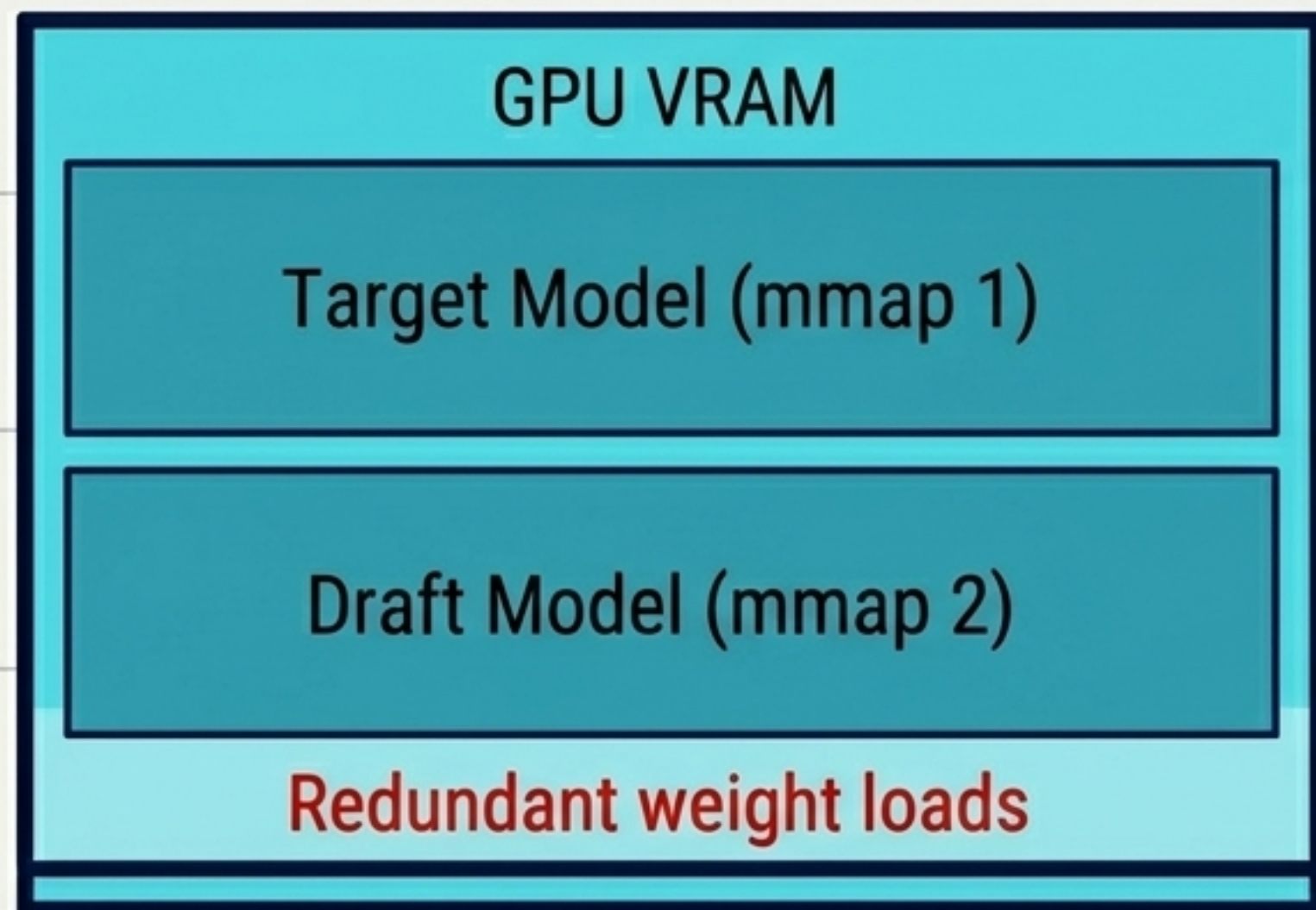
Multi-Token Prediction (MTP)



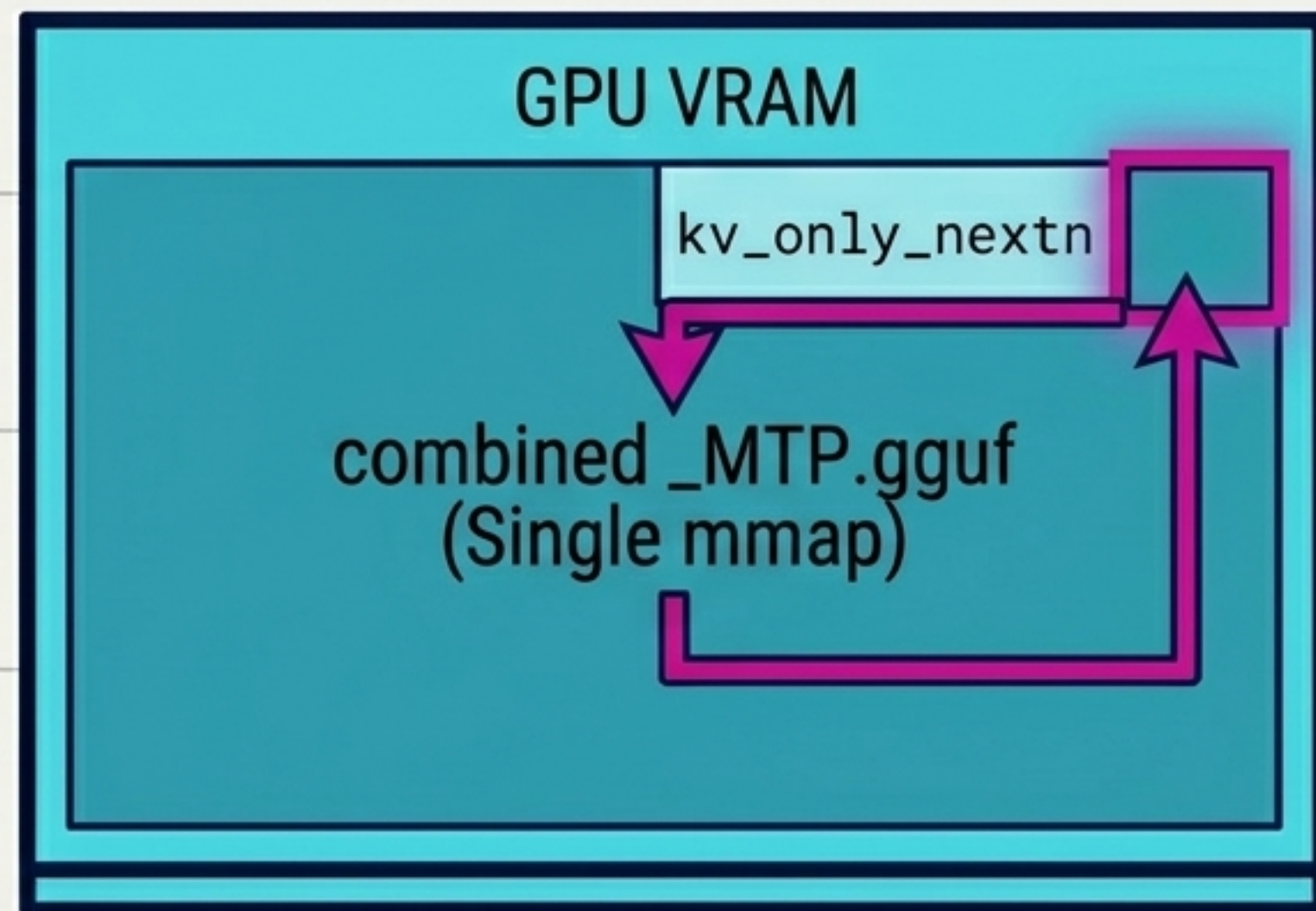
The shared-model draft path eliminates redundant memory mapping

Qwen 3.6 NextN speculative decoding reuses the already-loaded target model weights. No second mmap. No second tokenizer. Delivers +24-36% TPS on 35B-A3B MoE models.

The Old Way

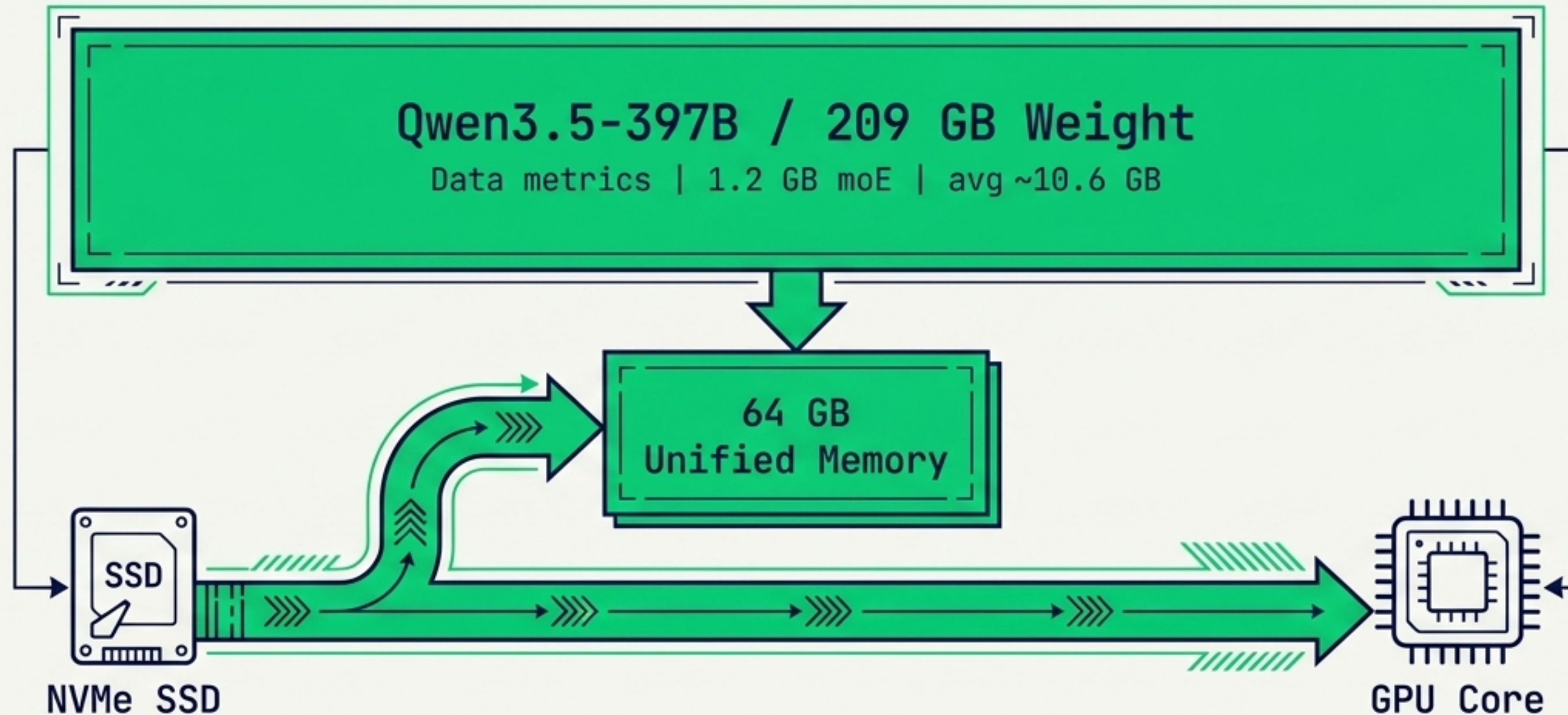


The New Way: NextN/MTP



Bypassing unified memory limits via SSD Expert Streaming

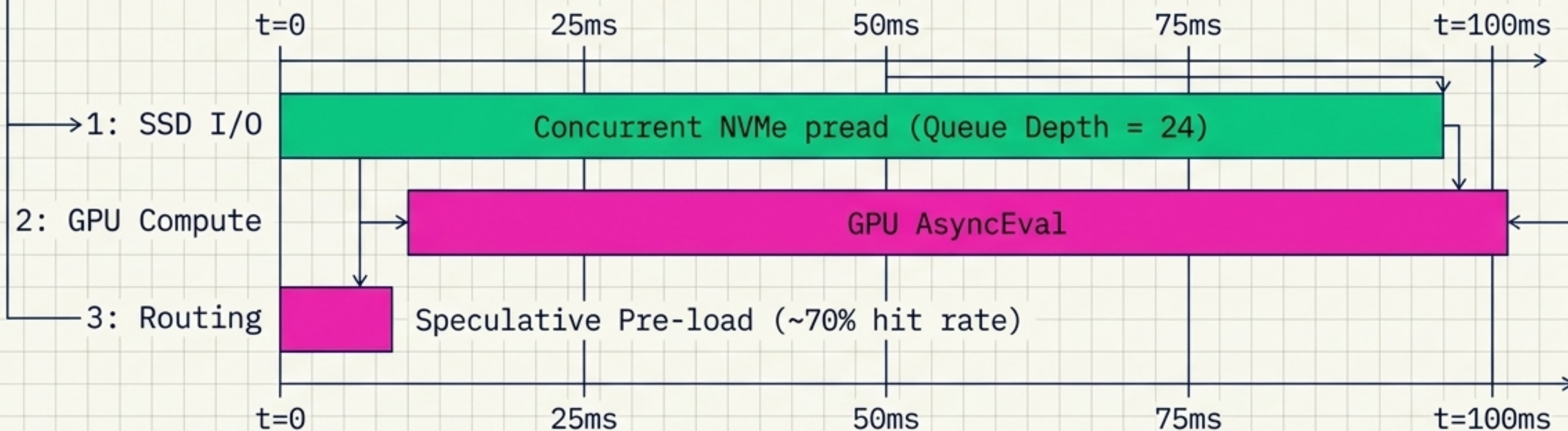
10x generation speedup on memory-constrained Apple Silicon. Evaluates **122B+** parameter MoE models using only **~10.6 GB** of resident memory.



Hiding SSD latency entirely behind GPU compute

Cross-projection batching collapses ~1,400 per-expert eval calls down to ~48, allowing the NVMe controller to saturate its queue depth while the GPU processes the previous token.

The Concurrent I/O Pipeline



The MoE Fan-Out Collision: When multipliers conflict

MTP evaluates multiple draft tokens in parallel. On massive MoE models running from SSD, this forces the system to fetch up to 3x as many unique experts simultaneously, saturating NVMe bandwidth and neutralizing MTP speedups. Solution: Cap draft tokens to 1 when streaming.

MTP: Parallel
Draft Tokens



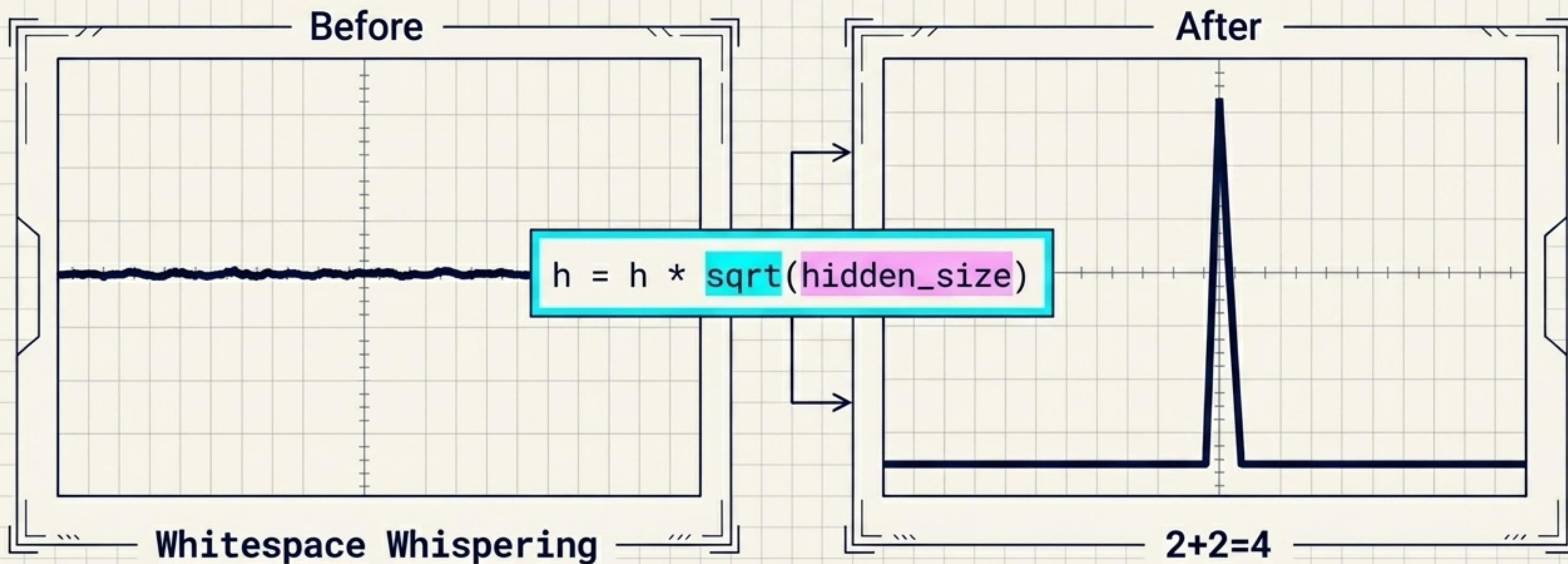
SSD Streaming:
Expert Weight
Fetches

Orchestrating the multipliers requires custom primitives

SwiftLM	Llama.cpp Forks	vLLM / SGLang
Focus: 100% Native Apple Silicon • Metal TurboFlash • AsyncEval Pipeline	Focus: Cross-Platform (Nvidia/AMD/Apple) • CUDA/HIP WHT kernels • Shared-model drafting	Focus: Datacenter & Production Serving • CUDA_SCALE_LAUNCH_QUEUES • Triton kernels

The bare-metal reality of custom inference engines

Missing a single embedding scale multiplier in the Gemma architecture means every activation is ~53x too small. One mathematical operation brings the entire pipeline online.



The multiplicative power of orthogonal breakthroughs

[AWQ/GGUF] × [TurboQuant]

Weight Quant

KV Quant

× [MTP] ×

Compute Speed

× [SSD Streaming] =

VRAM Bypass

**100B+ Parameter
Frontier Models
locally at 60+ tok/s**

The 2026 deployment playbook by hardware constraint

Hardware Deployment Decision Matrix

Apple M-Series (Unified Memory)	→ MLX-INT4 Weights + TurboQuant CPU/MPS + SwiftLM SSD Streaming
Nvidia RTX 5090 (24GB VRAM)	→ NVFP4 Weights + TurboQuant 3.5-bit + Llama.cpp MTP
AMD RX 7900 XTX (ROCm 6.4)	→ HIP port f16-K + Turbo-V mixed dispatch (SWA bypass)
Edge / Jetson Devices	→ Adaptive KV-Quant + TurboQuant 2.5-bit fallback

Frontier capabilities are now a local utility

Optimized software stacks and 3-bit cache compression have bridged the gap. Dual consumer GPUs are now actively driving **Sonnet-level coding** and massive RAG workloads with **complete privacy** and **zero API costs**.

```
def local_r.g_query(prompt): # Optimized local execution
    context = retrieve_documents(prompt)
    response = generate_response(context, model='Sonnet-Local',
                                quant='3-bit')
    return response
```

```
# RAG workload processing
results = local_r.g_query('Implement complex refactoring')
```

System:

2x RTX 3090
(48GB total)

113 tok/s
generation

4000 pp/s
prompt processing